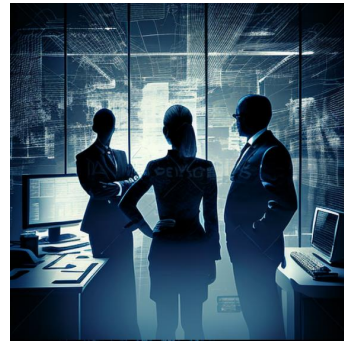


[Chap 16] 비즈니스 계획



선형계획법

선형계획법(LP: Linear Programming)

- 대표적인 최적화(optimization) 기법의 하나
- 선형계획모형(linear programming models)
 - 목적함수와 제약조건의 식이 모두 일차식
 - 변수들이 모두 실수값(real values)
 - 위의 조건에서 벗어나면 선형계획모형이 아님
 - 일차가 아닌 함수가 포함되면 비선형계획법(NLP: NonLinear Programming)
 - 정수(integer value)이어야만 하는 조건이 있으면 정수계획법(IP: Integer Programming)
- 선형계획모형은 수리계획모형(mathematical programming model)에서 매우 중요한 지위를 차지함
 - 수학적으로 잘 연구되어 있고 효율적인 최적해 도출 알고리즘도 다양
 - 단순한 모형이지만 응용 범위가 넓고 다른 복잡한 최적화 모형의 토대가 됨

3

선형계획 예제

한 빵집에서 케이크와 바게트를 판매하고 있다. 케이크 하나를 만드는 데에는 3컵의 밀가루가 필요하고, 오븐에서 45분간 구워야 한다. 바게트는 8컵의 밀가루가 필요하고, 오븐에서 30분간 구워야 한다. 그런데, 지금 빵집에는 20컵의 밀가루가 있고, 영업시작 전까지 3시간만 오븐을 이용할 수 있다. 케이크의 가격은 하나에 \$10이고, 바게트는 \$6이다. 오늘 빵집의 수익을 최대화하려면 케이크와 바게트를 몇 개씩 만들어야 하겠는가?

- 생산한 케이크와 바게트는 모두 팔린다고 가정
- 문제의 구조
 - 1) 케이크와 바게트의 생산량을 정해야 한다 → 결정변수(decision variable)
 - 2) 수익을 최대화하고 싶다 → 목적함수(objective function)
 - 3) 밀가루와 오븐시간이 제한되어 있다 → 제약조건(constraints)→ 선형계획모형의 세 가지 기본 요소

4

선형계획 예제 (계속)

- 결정변수 정의
 - X_1 : 케이크 생산량
 - X_2 : 바게트 생산량
- 목적함수 정의
 - 수익의 최대화가 목적임
 - 수익 = 케이크 가격 * 케이크 생산량 + 바게트 가격 * 바게트 생산량

Maximize $10 \cdot X_1 + 6 \cdot X_2$

- 제약조건 정의
 - 밀가루의 양과 오븐의 사용 시간에 제한되어 있음
 - 제품 생산량은 음수일 수 없음

subject to
 $3 \cdot X_1 + 8 \cdot X_2 \leq 20$ (밀가루 양 제약조건)
 $45 \cdot X_1 + 30 \cdot X_2 \leq 180$ (오븐 가동시간 제약조건)
 $X_1, X_2 \geq 0$

5

선형계획 예제 (계속)

- 선형계획모형 전체 정리

Maximize $10 \cdot X_1 + 6 \cdot X_2$

subject to

$3 \cdot X_1 + 8 \cdot X_2 \leq 20$ (밀가루 양 제약조건)

$45 \cdot X_1 + 30 \cdot X_2 \leq 180$ (오븐 가동시간 제약조건)

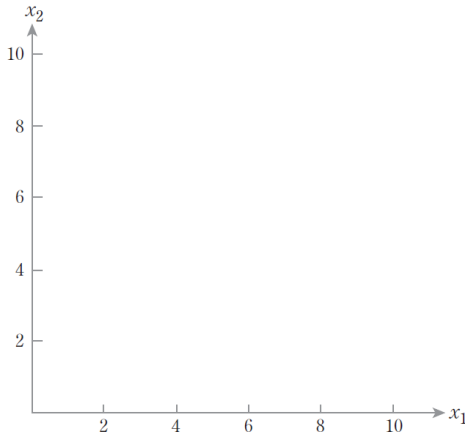
$X_1, X_2 \geq 0$

(여기서, X_1 = 케이크의 생산량, X_2 = 바게트의 생산량)

6

도해법을 이용한 최적해 계산

- 도해법(graphical solution method)
 - 각 결정변수를 좌표축으로 나타낸 공간에서 제약조건을 다각형으로 표시하여 제약 조건의 범위 안에 있는 '실행가능영역(feasible region)'을 구하고, 이 가능영역 내에서 최적해(optimal solution)를 찾는 방법
- 결정변수 각각을 좌표축으로 하는 공간
 - 예제에서는 변수가 두 개이므로 2차원 공간이 됨

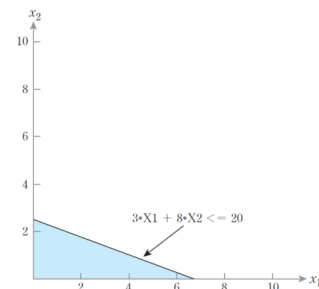


[그림 16-1] 결정변수 값의 좌표공간

7

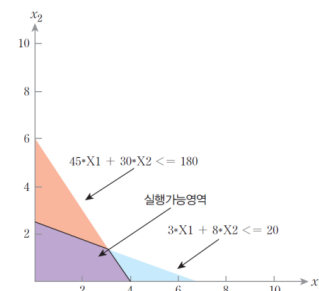
도해법을 이용한 최적해 계산 (계속)

- 밀가루 양의 제약조건
 - $3 \cdot x_1 + 8 \cdot x_2 \leq 20$ 의 영역을 좌표공간에 표현 (파란색으로 표시한 영역)



[그림 16-2] 밀가루 양 제약조건 충족 영역

- 오븐의 가동시간 제약조건
 - $45 \cdot x_1 + 30 \cdot x_2 \leq 180$ 의 영역을 표현 (빨간색으로 겹쳐 그림)



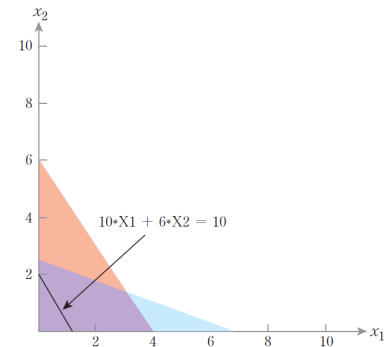
[그림 16-3] 모든 제약조건을 충족하는 실행가능영역

- 두 영역이 겹치는 부분(그림에서 보라색)
 - 모든 제약조건을 충족하여 실행가능영역이라고 함
 - 실행가능영역 내의 각각의 점들은 실행가능해(feasible solution)가 됨

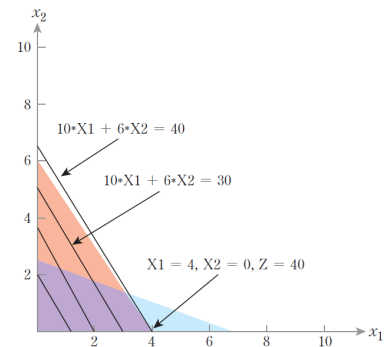
8

도해법을 이용한 최적해 계산 (계속)

- 가능해 영역 내의 실행가능해들 중에서 수익을 최대화하는 점을 찾으면 됨
- 먼저, 수익 \$10를 낼 수 있는 모든 가능한 점들은 그림 16-4의 검정색 선상에 있게 됨
 - \$10은 창출 가능한 해가 무수히 많음
- \$20, \$30, \$40을 창출할 수 있는 점이 있는지 검토
 - \$30까지는 창출 가능한 (X_1, X_2) 조합이 무수히 많음
 - \$40을 창출하는 선 중에서 가능해 영역 내에 있는 점은 $(4, 0)$ 뿐임
 - 그 이상 목표수익을 올리면 해당 선에서 가능해 영역 내에 있는 점이 없음
- 따라서, 최적해는 $X_1=4, X_2=0$ 이고 이 때의 수익은 \$40이 됨



[그림 16-4] 수익 \$10를 창출하는 실행가능해



[그림 16-5] 도해법에 의해 구해진 최적해

9

일반적인 선형계획모형의 최적해 도출

- 도해법은 변수 3개 이상부터는 적용이 어려움
 - 결정변수 4개부터는 기하학적인 표현 자체가 불가능
- 일반적 선형계획모형의 풀이
 - 심플렉스법(simplex method) : 기하학적 그래프에 의존하지 않고 대수적으로 선형계획모형의 최적해를 구하는 기법
 - 이외에도 여러 기법이 있음
 - 최적해를 계산해주는 다양한 소프트웨어 활용

PuLP로 선형계획모형 최적해 도출

- PuLP 라이브러리: 파이썬에서 선형계획모형의 최적해 계산 지원
 - PuLP의 설치: `pip install pulp`

```
명령 프롬프트
Microsoft Windows [Version 10.0.19044.1826]
(c) Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>pip install pulp_
```

- pulp의 모든 함수명을 그냥 사용할 수 있도록 import

```
from pulp import *
```

- LpProblem 명령어로 선형계획모형을 생성하여 임의의 이름을 정하고, 최대화인지 최소화인지를 설정
 - 목표가 최대화이면 'LpMaximize', 최소화라면 'LpMinimize'로 설정

```
LP = LpProblem("bakery_problem", LpMaximize)
```

11

PuLP로 선형계획모형 최적해 도출 (계속)

- LpVariable 명령어로 결정변수를 설정

```
1 X1 = LpVariable("cake")
2 X2 = LpVariable("baguette")
```

- 목적함수 설정
 - 앞에서 모형을 생성하여 저장했던 변수 LP에 다음과 같이 목적함수 식을 추가

```
LP += 10*X1 + 6*X2
```

- 제약조건 추가
 - 부등호까지 함께 입력함에 주의

```
1 LP += 3*X1 + 8*X2 <= 20
2 LP += 45*X1 + 30*X2 <= 180
3 LP += X1 >= 0
4 LP += X2 >= 0
```

12

PuLP로 선형계획모형 최적해 도출 (계속)

- `.solve()` : 문제를 풀고 최적해를 도출

```
LP.solve()
```

```
Optimal - objective value 40
After Postsolve, objective 40, infeasibilities - dual 0 (0), primal 0 (0)
Optimal objective 40 - 1 iterations time 0.002, Presolve 0.00
Option for printingOptions changed from normal to all
Total time (CPU seconds):      0.01  (Wallclock seconds):      0.00
```

- 최종 목적함수값이 40으로서 도해법의 풀이 결과와 동일
- 최적해에서 각 변수의 값과 목적함수값 확인
 - `LP.variables()`의 각 요소에서 이름(`.name`)과 값(`.varValue`) 및 목적함수값(`.objective`) 출력

```
1 for v in LP.variables():
2     print(v.name, '=', v.varValue)
3 print("Total profit is ", value(LP.objective))
```

13

PuLP로 선형계획모형 최적해 도출 (계속)

- 전체 코드와 출력 결과

예제 ex16-1.py

```
1 from pulp import *
2
3 LP = LpProblem("bakery_problem", LpMaximize)
4 X1 = LpVariable("cake")
5 X2 = LpVariable("baguette")
6 LP += 10*X1 + 6*X2
7 LP += 3*X1 + 8*X2 <= 20
8 LP += 45*X1 + 30*X2 <= 180
9 LP += X1 >= 0
10 LP += X2 >= 0
11 LP.solve()
12 for v in LP.variables():
13     print(v.name, '=', v.varValue)
14 print("Total profit is ", value(LP.objective))
```

```
baguette = 0.0
cake = 4.0
Total profit is 40.0
```

선형계획법을 이용한 의사결정 예제

‘행복한 소시지’는 소시지를 만드는 기업이다. 이 공장은 닭고기, 소고기, 돼지고기, 전분을 배합해 소시지를 만든다. 재료들은 하루에 닭고기 200파운드, 소고기 300파운드, 돼지고기 150파운드, 전분 400파운드가 들어오고, 각 원료의 가격은 파운드 당 \$0.2, \$0.3, \$0.5, \$0.05이다.

‘행복한 소시지’는 원료의 배합에 따라 일반 소시지, 소고기 소시지, 고기만 소시지를 판매한다. 일반 소시지는 소고기와 돼지고기를 합친 비율이 10% 이하이고, 닭고기의 비율이 20% 이상이어야 한다. 소고기 소시지는 소고기의 비율이 75% 이상이어야 한다. 고기만 소시지는 전분이 들어가지 않고, 소고기와 돼지고기를 합친 비율이 50% 이하여야 한다.

일반 소시지는 파운드 당 \$0.9, 소고기 소시지는 \$1.25, 고기만 소시지는 \$1.75에 판매된다. 이때 ‘행복한 소시지’의 수익이 최대화되는 생산량을 구하시오.

15

선형계획법을 이용한 의사결정 예제 (계속)

- 결정변수: 각 원료가 각 소시지의 생산에 투입되는 수량
 - X_{ij} (i = 원료, j = 소시지)의 형태로 표현

X_{ij} = 각 원료가 소시지의 생산에 투입되는 수량
 $i = c, b, p, s$ (c : chicken, b : beef, p : pork, s : starch)
 $j = r, b, m$ (r : regular, b : beef, m : meat)

- 목적함수 : 수익의 최대화
 - 단위당 수익은 각 소시지의 가격에서 원료의 가격을 뺀 값

Maximize $0.7 \cdot X_{cr} + 0.6 \cdot X_{br} + 0.4 \cdot X_{pr} + 0.85 \cdot X_{ar}$
 $+ 1.05 \cdot X_{cb} + 0.95 \cdot X_{bb} + 0.75 \cdot X_{pb} + 1.20 \cdot X_{ab}$
 $+ 1.55 \cdot X_{cm} + 1.45 \cdot X_{bm} + 1.25 \cdot X_{pm} + 1.70 \cdot X_{am}$

16

선형계획법을 이용한 의사결정 예제 (계속)

- 제약조건
 - 각 원료의 수량 제약과 소시지 원료 비율에 대한 제약으로 구성

```
subject to
Xcr + Xcb + Xcm ≤ 200 (닭고기의 수량 조건)
Xbr + Xbb + Xbm ≤ 300 (소고기의 수량 조건)
Xpr + Xpb + Xpm ≤ 150 (돼지고기의 수량 조건)
Xar + Xab + Xam ≤ 400 (전분의 수량 조건)
0.9*Xbr + 0.9*Xpr - 0.1*Xcr - 0.1*Xar ≤ 0 (일반 소시지의 비율 조건)
0.8*Xcr - 0.2*Xbr - 0.2*Xpr - 0.2*Xar ≥ 0 (일반 소시지의 비율 조건)
0.25*Xbb - 0.75*Xcb - 0.75*Xpb - 0.75*Xab ≥ 0 (소고기 소시지의 비율 조건)
Xam = 0 (고기만 소시지의 비율 조건)
0.5*Xbm + 0.5*Xpm - 0.5*Xcm - 0.5*Xam ≤ 0 (고기만 소시지의 비율 조건)
Xij ≥ 0
```

17

선형계획법을 이용한 의사결정 예제 (계속)

- 전체 선형계획모형

```
Maximize 0.7*Xcr + 0.6*Xbr + 0.4*Xpr + 0.85*Xar
          + 1.05*Xcb + 0.95*Xbb + 0.75*Xpb + 1.20*Xab
          + 1.55*Xcm + 1.45*Xbm + 1.25*Xpm + 1.70*Xam

subject to
Xcr + Xcb + Xcm ≤ 200 (닭고기의 수량 조건)
Xbr + Xbb + Xbm ≤ 300 (소고기의 수량 조건)
Xpr + Xpb + Xpm ≤ 150 (돼지고기의 수량 조건)
Xar + Xab + Xam ≤ 400 (전분의 수량 조건)
0.9*Xbr + 0.9*Xpr - 0.1*Xcr - 0.1*Xar ≤ 0 (일반 소시지의 비율 조건)
0.8*Xcr - 0.2*Xbr - 0.2*Xpr - 0.2*Xar ≥ 0 (일반 소시지의 비율 조건)
0.25*Xbb - 0.75*Xcb - 0.75*Xpb - 0.75*Xab ≥ 0 (소고기 소시지의 비율 조건)
Xam = 0 (고기만 소시지의 비율 조건)
0.5*Xbm + 0.5*Xpm - 0.5*Xcm - 0.5*Xam ≤ 0 (고기만 소시지의 비율 조건)
Xij ≥ 0

(Xij = 각 원료가 소시지의 생산에 투입되는 수량. 여기서
i = c, b, p, s (c: chicken, b: beef, p: pork, s: starch),
j = r, b, m (r: regular, b: beef, m: meat) )
```

18

PuLP로 선형계획법 최적해 도출

- Pulp를 import하고, 문제의 이름을 'sausage_problem'으로 정의하고, 목표는 최대화로 설정

```
1 from pulp import *
2 LP = LpProblem("sausage_problem", LpMaximize)
```

- 결정변수 정의
 - 결정변수의 하한값을 0으로 주어서 음수 생산량 방지

```
1 Xcr = LpVariable("Xcr",0)
2 Xbr = LpVariable("Xbr",0)
3 Xpr = LpVariable("Xpr",0)
4 Xar = LpVariable("Xar",0)
5 Xcb = LpVariable("Xcb",0)
6 Xbb = LpVariable("Xbb",0)
7 Xpb = LpVariable("Xpb",0)
8 Xab = LpVariable("Xab",0)
9 Xcm = LpVariable("Xcm",0)
10 Xbm = LpVariable("Xbm",0)
11 Xpm = LpVariable("Xpm",0)
12 Xam = LpVariable("Xam",0)
```

19

PuLP로 선형계획법 최적해 도출 (계속)

- 목적함수 입력
 - 파이썬에서 한 명령어를 여러 줄로 사용할 때는 줄 끝에 '\ '기호 추가

```
1 LP += 0.7*Xcr + 0.6*Xbr + 0.4*Xpr + 0.85*Xar \
2     + 1.05*Xcb + 0.95*Xbb + 0.75*Xpb + 1.20*Xab \
3     + 1.55*Xcm + 1.45*Xbm + 1.25*Xpm + 1.70*Xam
```

- 제약조건 추가
 - 등호, 부등호도 표현함. 등호는 파이썬에서 '=='로 표시

```
LP += Xcr + Xcb + Xcm <= 200
LP += Xbr + Xbb + Xbm <= 300
LP += Xpr + Xpb + Xpm <= 150
LP += Xar + Xab + Xam <= 400
LP += 0.9*Xbr + 0.9*Xpr - 0.1*Xcr - 0.1*Xar <= 0
LP += 0.8*Xcr - 0.2*Xbr - 0.2*Xpr - 0.2*Xar >= 0
LP += 0.25*Xbb - 0.75*Xcb - 0.75*Xpb - 0.75*Xab >= 0
LP += Xam == 0
LP += 0.5*Xbm + 0.5*Xpm - 0.5*Xcm - 0.5*Xam <= 0
```

20

PuLP로 선형계획법 최적해 도출 (계속)

- 모형 최적화 및 변수값 출력 (전체 코드: ex16-2.py)

```
1 LP.solve()
2
3 for v in LP.variables():
4     print(v.name, '=', v.varValue)
5 print("Total profit is ", value(LP.objective))
```

```
Xab = 100.0
Xam = 0.0
Xar = 300.0
Xbb = 300.0
Xbm = 0.0
Xbr = 0.0
Xcb = 0.0
Xcm = 125.0
Xcr = 75.0
Xpb = 0.0
Xpm = 125.0
Xpr = 0.0
Total profit is 1062.5
```

21

뉴스벤더 모델

뉴스벤더 모델(newsvendor model) 개요

- 신문판매원 모형, 단일기간재고모형 등으로 번역
- 매우 단순하지만 공급망의 생산판매계획(S&OP: Sales and Operations Planning) 수립 등 대규모 의사결정 시스템의 근간을 형성하는 중요한 모형
- 뉴스벤더 모델의 기본적 가정
 - 수요는 일정기간동안만 발생한다.
 - 수요기간동안의 수요량은 확률적이다.
 - 생산은 수요기간 이전에 완료되어야 하며, 수요기간동안 추가 보충은 없다.
 - 수요기간동안 제품은 생산비보다 높은 가격으로 판매된다.
 - 수요기간이 끝나고 남은 잔존재고는 생산비용보다 낮은 가격으로 전량 처분된다

23

뉴스벤더 모델 개요 (계속)

- 재고부족(understock)과 재고과잉(overstock) 상황의 비대칭
 - 재고부족비용(understock cost): 재고가 모자라게 되면 판매기회상실로 이어지므로, 추가 수요마다 제품 판매가와 생산비의 차이만큼의 기회비용이 발생
 - 재고과잉비용(overstock cost): 수요기간이 끝날 때까지 남아있는 재고는 생산비 이하로 처분해야 하므로 잔존재고 단위당 생산비와 잔존가격 차이만큼 손실 발생
- 제품 개당 정상판매가를 p , 생산비를 c , 잔존가격을 s 라고 하면,
 - 재고부족비용: $C_u = p - c$
 - 재고과잉비용: $C_o = c - s$
- 뉴스벤더 모델의 예
 - 새벽에 신문배급소에서 개당 100원에 신문을 사온다고 가정($c = 100$)
 - 오늘 하루동안은 신문을 500원에 팔 수 있으나($p = 500$), 오늘이 다 갈때까지 남은 신문은 그냥 버려야 함($s = 0$)

$$C_u = p - c = 500 - 100 = 400$$

$$C_o = c - s = 100 - 0 = 100$$

- 수요는 평균 300부로 예측: 300부보다 많을 수도, 적을 수도 있음

뉴스벤더 모델 개요 (계속)

- 뉴스벤더 의사결정의 구조
 - 수요는 300보다 많을 수도 적을 수도 있음
 - 만약 신문이 다 팔리고 없을 때 또 다른 손님이 오면 추가 보충이 불가능하므로 400원의 이익을 잃음 → 모자라면 400원 손실
 - 반대로, 오늘이 다 가기까지 안 팔린 신문은 그냥 버려야 하므로 개당 100원의 손실이 발생 → 남으면 100원 손실
 - 모자라는 것이 더 큰 손실이므로, 약간 남을 정도로 신문을 사 오는 것이 합리적 → 주문량은 300보다 큰 것이 좋음
 - 참고로, 실제 사람들을 대상으로 모의실험을 해보면 합리적인 최적보다는 수요예측에 맞추는 방향으로, 즉, 평균 수요량 예측만큼 주문하려는 경향이 있음
- 수요는 평균이 300, 표준편차가 150인 정규분포를 따른다고 가정
 - 정확히 몇 부를 주문해야 하는지 결정하려면 수요의 확률분포 예측 필요
 - 수요량을 x , 수요의 확률밀도함수를 $f(x)$ 라고 하면, 주문량(생산량)이 Q 일 때의 기대이익 $\Pi(Q)$ 는
$$\Pi(Q) = -cQ + \int_0^Q \{px + s(Q-x)\}f(x)dx + \int_Q^\infty pQf(x)dx$$
 - 이 식을 전개하여 정확한 이익을 계산할 수 있으나, 파이썬을 통한 시뮬레이션으로 이익을 계산해보기로 함

25

뉴스벤더 모델 시뮬레이션

- 관련 비용과 C_u, C_o 설정

```
1 import numpy as np
2 p=500
3 c=100
4 s=0
5 Cu=p-c
6 Co=c-s
```

- 수요량 생성
 - 평균을 μ , 표준편차를 σ 라고 할 때, 해당 평균과 표준편차를 가지는 정규분포를 따르는 수요량 x 는 $\text{np.random.normal}(\mu, \sigma)$ 로 발생시킬 수 있음

```
1 mu=300
2 sigma=150
3 x = np.random.normal(mu, sigma)
4 if x<0: x=0
```

- (4행) 만약 수요가 음수이면 0으로 만듦

26

뉴스벤더 모델 시뮬레이션 (계속)

- 수요가 x 일 때의 이익 계산

```
1 profit = -c*Q
2 if x<=Q: profit += p*x + s*(Q-x)
3 else: profit += p*Q
```

- (1행) 초기 구매비로 $c*Q$ 만큼 지출
- (2행) 만약 수요 x 가 주문량 Q 이하이면 x 개에 대해 개당 p 의 매출 발생, 잔존재고 $Q-x$ 만큼에 대해 개당 s 의 잔존가 회수
- (3행) 만약 수요가 주문량을 초과하면 주문량 Q 개까지만 팔 수 있고, 이에 대한 개당 p 의 매출 발생. 잔존량은 없음

27

뉴스벤더 모델 시뮬레이션 (계속)

- 위의 상황을 100,000회 반복하면서 평균 이익을 출력
 - 주문량 $Q=300$ 으로 하면 기대이익이 약 90725원으로 계산됨
 - 난수로 계산하므로 실행할 때마다 약간의 차이 발생

```
Order Quantity = 300
Expected profit = 90725.46723097205
```

- 주문량을 바꾸어 가면서 이익이 최대화되는 주문량을 찾을 수 있음

예제 ex16-3.py

```
1 import numpy as np
2 p=500
3 c=100
4 s=0
5 Cu=p-c
6 Co=c-s
7 mu=300
8 sigma=150
9
10 Q=300
11 n=100000
12 sum = 0
13 for i in range(n):
14     x = np.random.normal(mu, sigma)
15     if x<0: x=0
16     profit = -c*Q
17     if x<=Q: profit += p*x + s*(Q-x)
18     else: profit += p*Q
19     sum += profit
20 avg = sum/n
21 print("Order Quantity = ", Q)
22 print("Expected profit = ", avg)
```

28

뉴스벤더 모델의 최적 주문량

- 뉴스벤더 모델의 최적 서비스수준(임계율)
 - 수요량 전체 중에서 임계율만큼을 대응하는 것이 최적이라는 의미

$$\alpha = \frac{C_u}{C_o + C_u}$$

- 예) 앞의 신문팔이 소년의 경우, 수요의 80%를 충족하는 주문량을 결정하는 것이 최적

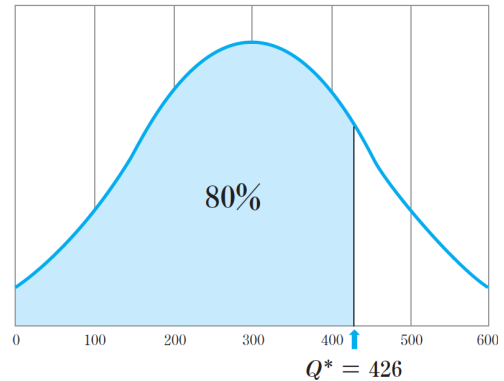
$$\alpha = \frac{C_u}{C_o + C_u} = \frac{400}{100 + 400} = 0.8$$

- 최적 주문량 결정
 - 확률밀도함수 곡선 아래 누적 면적이 임계율과 같아지는 주문량을 찾으면 됨
 - 엑셀의 NORM.INV 함수 사용
 - 수요량이 평균 300, 표준편차 150인 정규분포를 따른다면 NORM.INV(0.8, 300, 150) → 426이 됨

- 앞에서 구현한 시뮬레이터에 Q=426으로 하면 기대이익이 약 99650원으로 10% 이상 증가

Order Quantity = 426
Expected profit = 99649.7226621867

- 의사결정에서 수요 변동성 고려의 중요성 확인



[그림 16-6] 최적 주문량의 결정

29

인공신경망 적용을 위한 데이터 생성

- 수요가 순수하게 확률적으로 발생하는 것이 아니라 다른 외부요인들로부터 영향을 받는 경우를 가정
- 복잡한 영향요인이 있는 상황에서 수요를 예측하는 데 인공신경망 활용 가능
 - 인공신경망은 특성값과 결과값 사이의 숨어있는 패턴을 찾아서 근사함수를 생성
 - 그런데, 단순히 수요의 값을 예측하는 것으로는 부족하므로, 인공신경망이 수요가 아니라 최적 생산량을 바로 예측하도록 학습시키는 방법 필요
 - Oroojlooyjadid 등(2020)에서 제시된 모형을 응용

- 냉면 가게 예제
 - 냉면의 수요는 기온이 높아질수록 증가하고, 온도가 t도일 때 이 식당의 냉면의 수요는 평균 $5t+50$ 으로 나타난다고 가정
 - 또한, 냉면의 수요에는 변동성이 있는데 평소에는 평균수요 대비 약 $\pm 50\%$ 의 변동성이 있으나 날씨가 비가 오면 변동성이 $\pm 60\%$ 로 증가
 - 기온 t, 날씨 w(w=0이면 보통, 1이면 비)라고 하면, 일일 수요 d는

$$\mu_d = 5t + 50$$

$$\sigma_d = \begin{cases} 0.5\mu_d, & \text{if } w = 0 \\ 0.6\mu_d, & \text{if } w = 1 \end{cases}$$

인 정규분포로 나타남

30

인공신경망 적용을 위한 데이터 생성 (계속)

- 기온은 0도와 30도 사이에서 균일한 확률로 랜덤하게 나온다고 가정하고, 날씨는 보통 ($w=0$)일 확률이 70%, 비가 올($w=1$) 확률이 30%일 때, 냉면 가게의 수요를 10건 생성
 - (3~4행) 기온을 temp, 날씨를 weather에 각각 n개씩의 균등분포와 이항분포 값을 만들어 넣음
 - (5행) 수요를 저장할 배열 demand를 n개의 요소로 초기화
 - (7행) 각각의 요소에 대해 수요의 평균 계산
 - (8~9행) 날씨가 0인지 1인지에 따라 수요의 표준편차를 계산
 - (10행~12행) 평균과 표준편차에 따라 정규분포 난수를 만들어 demand[i]에 저장
 - 결과는 다음과 같이 차례로 기온, 날씨, 해당 수요량이 생성되어 출력
 - 난수이므로 차이가 있을 수 있음

```
[ 2.1806825  0.75857485  8.32267316  9.05014197 16.89969309 23.35055747
19.74189414 16.72264722  8.50543526  4.31712682]
[0 0 1 0 1 1 0 0 1 0]
[ 83.88534  27.00036 112.39334  48.647255 25.693317 103.13888
176.96065 102.95215 115.89846  56.839222]
```

```
1 import numpy as np
2 n=10
3 temp = np.random.uniform(0, 30, n)
4 weather = np.random.binomial(1, 0.3, n)
5 demand = np.zeros(n, dtype='float32')
6 for i in range(n):
7     mu_d = 5*temp[i] + 50
8     if weather[i]==0: sigma_d = 0.5 * mu_d
9     else: sigma_d = 0.6 * mu_d
10    d = np.random.normal(mu_d, sigma_d)
11    if d<0: d=0
12    demand[i] = d
13 print(temp)
14 print(weather)
15 print(demand)
```

31

평균수요만큼 생산했을 때의 시뮬레이션

- 비즈니스의 설정
 - 냉면 한 그릇의 원가는 2000원, 판매가는 8000원이라고 가정
 - 은 냉면은 폐기처분하여 잔존가는 0이라고 가정
 - 평균수요만큼 생산할 때의 기대이익 시뮬레이션 → 주문량 125, 기대이익 약 545,344원

예제 ex16-4.py

수요 생성

```
1 import numpy as np
2 n=100000
3 temp = np.random.uniform(0, 30, n)
4 weather = np.random.binomial(1, 0.3, n)
5 demand = np.zeros(n, dtype='float32')
6 for i in range(n):
7     mu_d = 5*temp[i] + 50
8     if weather[i]==0: sigma_d = 0.5 * mu_d
9     else: sigma_d = 0.6 * mu_d
10    d = np.random.normal(mu_d, sigma_d)
11    if d<0: d=0
12    demand[i] = d
13
14 p=8000
15 c=2000
16 s=0
17 Cu=p-c
18 Co=c-s
```

```
19
20 sum_profit = 0
21 sum_Q = 0
22 for i in range(n):
23     mu_d = 5*temp[i] + 50
24     Q = mu_d
25     d = demand[i]
26     profit = -c*Q
27     if d<=Q: profit += p*d + s*(Q-d)
28     else: profit += p*Q
29     sum_Q += Q
30     sum_profit += profit
31
32 avg_Q = sum_Q/n
33 avg_profit = sum_profit/n
34 print("Production Quantity = ", avg_Q)
35 print("Expected profit = ", avg_profit)
```

Q를 평균수요
만큼으로 했을
때의 이익 계산

32

인공신경망을 활용한 생산계획 최적화 구현

- 생산계획 최적화에 인공신경망 적용
 - 손실함수에 비용의 비대칭성을 고려(Oroojlooyjadid 등, 2020)
- 일반적 손실함수의 한계
 - 생산량 예측은 연속값이므로 일반적으로는 손실함수로서 평균제곱오차(mean square error) 등을 사용하나, 이는 정답값에 비해 예측값이 큰 경우와 작은 경우의 손실이 동일함(대칭적)
 - 따라서 이 손실함수값을 최소화하는 방향으로 학습하면 나중에 이 신경망은 학습한 정답값에 최대한 가까운 값을 예측하는 방향으로 최적화되어 수요를 예측하게 됨
 - 예를 들어 이 신경망에 기온과 날씨를 주고 수요를 학습시켰다면 나중에 기온과 날씨에 대응하는 수요량을 비슷하게 예측하게 될 것임
- 인공신경망이 수요를 예측하는 대신 최적생산량을 산출하는 방향으로 학습시키려면 손실함수가 바뀌어야 함
 - 신경망이 결정한 생산량이 수요보다 크면 재고과잉비용 C_o 에 재고과잉량을 곱한 값이, 생산량이 수요보다 작으면 재고부족비용 C_u 에 재고부족량을 곱한 값이 손실이 되어야 함

$$Loss(Q, d) = \begin{cases} C_o \cdot |Q - d|, & \text{if } Q \geq d \\ C_u \cdot |Q - d|, & \text{if } Q < d \end{cases}$$

33

인공신경망을 활용한 생산계획 최적화 (계속)

- 맞춤형 손실함수(custom loss function) 구현

```
1 p=8000
2 c=2000
3 s=0
4 Cu=p-c
5 Co=c-s
6 import tensorflow as tf
7 def inventory_loss(demand_true, q):
8     stock = q-demand_true
9     is_understock = stock < 0
10    inventory_error = tf.abs(stock)
11    understock_loss = Cu*inventory_error
12    overstock_loss = Co*inventory_error
13    return tf.where(is_understock, understock_loss, overstock_loss)
```

- (8행) 생산량에서 실현 수요를 빼서 stock에 저장
- (9행) is_understock은 stock이 음수일 때(재고부족상태일때) 1, 아니면 0의 값을 갖게 됨
- (10행) 생산량과 실현수요량의 차이의 절대값을 구해서 inventory_error로 둠
- (13행) tf.where는 첫 인자로 들어온 값이 1이면 두 번째 인자를, 아니면 세 번째 인자를 리턴
 - 따라서 재고부족상태이면 understock_loss의 값이, 재고과잉상태이면 overstock_loss의 값이 리턴됨

34

인공신경망을 활용한 생산계획 최적화 (계속)

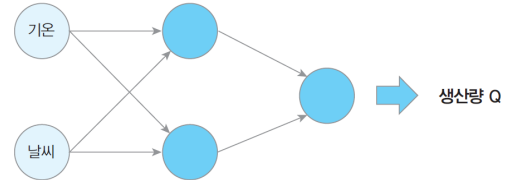
- 학습 데이터 생성

```
1 x_train = np.zeros((n,2), dtype='float32')
2 x_train[:,0] = (temp - min(temp))/(max(temp)-min(temp))
3 x_train[:,1] = weather
4 demand_train = (demand - min(demand))/(max(demand)-min(demand))
```

- (2행) 기온 temp를 0과 1 사이로 규모를 조정하여 x_train의 첫번째 열에 넣음
- (3행) 날씨 weather를 x_train의 두번째 열에 넣음
- (4행) 정답값 수요 데이터를 demand_train에 넣음

- 신경망 구성

- 입력 데이터 2개 (기온, 날씨)
- 출력층은 연속값(생산량) 출력
- 은닉층은 단일 계층, 2개의 뉴런을 둬



[그림 16-7] 생산량을 도출하는 인공신경망 구조

```
1 from tensorflow import keras
2 model = keras.Sequential([
3     keras.layers.Dense(2, input_shape=(2,), activation='sigmoid'),
4     keras.layers.Dense(1, activation=None),
5 ])
```

35

인공신경망을 활용한 생산계획 최적화 (계속)

- 최적화알고리즘과 손실함수를 정하고 학습 수행

- 이 때 앞에서 구현한 맞춤형 손실함수인 inventory_loss로 손실함수를 지정

```
1 model.compile(optimizer='adam', loss=inventory_loss)
2 model.fit(x_train, demand_train, epochs=20)
```

- 테스트 데이터를 10000건 구성하여 산출된 생산량의 평균을 출력

- 신경망이 결정한 생산량은 평균수요 125와는 큰 차이가 있음

```
1 n_test=10000
2 temp_new = np.random.uniform(0, 30, n_test)
3 weather_new = np.random.binomial(1, 0.3, n_test)
4 x_test = np.zeros((n_test,2), dtype='float32')
5 x_test[:,0]=(temp_new - min(temp))/(max(temp)-min(temp))
6 x_test[:,1]=np.random.binomial(1, 0.3, n_test)
7 pred = model.predict(x_test)
8 Q_prop = pred*(max(demand)-min(demand))+min(demand)
9 print("Mean of Q_prop = ", np.mean(Q_prop))
```

Mean of Q_prop = 169.59798

36

인공신경망 생산계획의 성능 시뮬레이션

- 앞서 작성한 코드에는 테스트용 입력데이터 `x_test`와 이것에 대응하는 신경망의 생산량 출력값 `Q_prop`이 산출되어 있으므로, 이 값으로 성능 시뮬레이션
 - (4~6행) 기온과 날씨에 따라 수요의 평균과 표준편차 계산
 - (7~8행) 평균과 표준편차에 따라 수요량 생성
 - (9~12행) 해당 수요 `d`와 신경망 산출 생산량 `Q_prop`에 대응하는 이익 계산

- 실행 결과

```
Production Quantity = [170.34726]
Expected profit = [587996.61056]
```

- 평균 생산량 170, 기대이익 587,997원
- 생산량을 수요량 평균과 동일하게 하였을 때 기대이익은 약 545,344원이었음
 - 이익 개선율 약 7.8%

```
1 sum_profit = 0
2 sum_Q = 0
3 for i in range(n_test):
4     mu_d = 5*temp_new[i] + 50
5     if weather_new[i]==0: sigma_d = 0.5 * mu_d
6     else: sigma_d = 0.6 * mu_d
7     d = np.random.normal(mu_d, sigma_d)
8     if d<0: d=0
9     Q = Q_prop[i]
10    profit = -c*Q
11    if d<=Q: profit += p*d + s*(Q-d)
12    else: profit += p*Q
13    sum_Q += Q
14    sum_profit += profit
15 avg_Q = sum_Q/n
16 avg_profit = sum_profit/n
17 print("Production Quantity = ", avg_Q)
18 print("Expected profit = ", avg_profit)
```